DECODELABS INDUSTRIAL TRAINING

Project 3: CI/CD Pipeline Basics Using GitHub Actions

Prepared by:
Chiemezuo Martina

Date:
July 2026

Table of Contents

---

1. Introduction

Continuous Integration (CI) and Continuous Deployment (CD) are fundamental DevOps practices that automate the software delivery process. Instead of manually building, testing, and deploying applications, developers configure automated pipelines that execute these tasks whenever code changes are pushed to a repository.

In this project, GitHub Actions was used to build a simple CI pipeline. The workflow automatically runs whenever code is pushed to the GitHub repository, demonstrating the core principles of automation in modern software development.

---

2. Project Objectives

The objectives of this project were to:

- Understand Continuous Integration (CI).
- Understand Continuous Deployment (CD).
- Create an automated GitHub Actions workflow.
- Trigger a pipeline automatically after a Git push.
- Execute automated workflow steps.
- Monitor workflow execution through GitHub Actions.

---

3. What is CI/CD?

Continuous Integration (CI)

Continuous Integration is the practice of automatically building and testing code whenever developers submit changes to a repository. This helps detect issues early and ensures that new code integrates correctly with the existing project.

Continuous Deployment (CD)

Continuous Deployment is the practice of automatically delivering verified code to production or another deployment environment after successful testing. It reduces manual work and enables faster software releases.

---

4. What is GitHub Actions?

GitHub Actions is GitHub's built-in automation platform that allows developers to create workflows directly inside a repository.

Workflows are written using YAML files and stored inside:

.github/workflows/

Whenever a specified event occurs, such as pushing code to the repository, GitHub automatically executes the workflow.

---

5. Environment Setup

Component	Description
Operating System	Ubuntu 24.04.4 LTS (WSL)
Terminal	Ubuntu Terminal
Version Control	Git
Remote Repository	GitHub
CI/CD Platform	GitHub Actions

---

6. Practical Exercises

Task 1 – Create the GitHub Actions Directory

Objective

To create the folder structure required by GitHub Actions.

Commands Used

mkdir -p .github/workflows

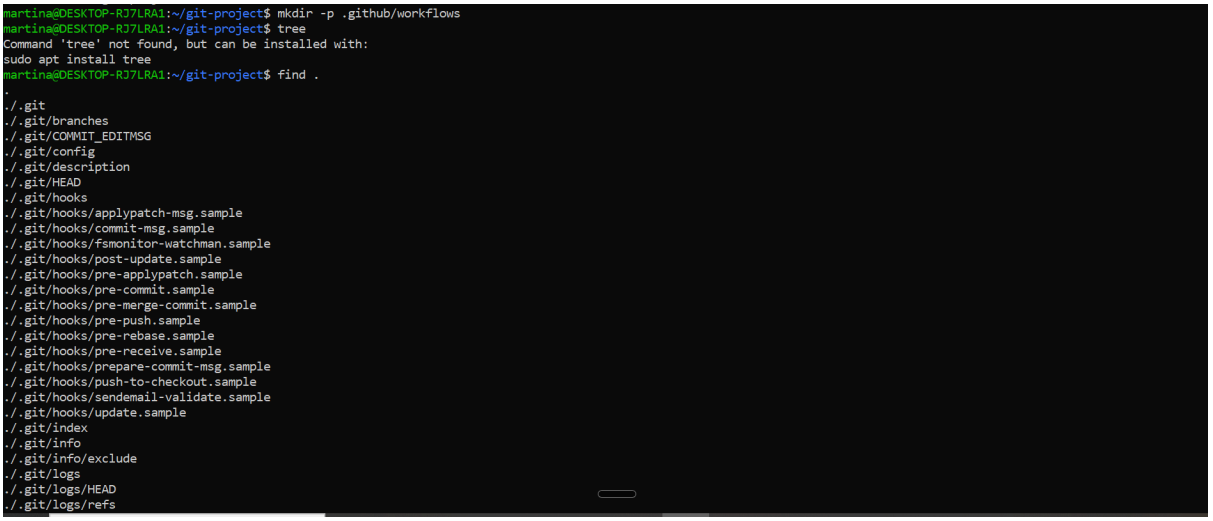
find .

Explanation

The `mkdir -p` command creates the hidden `.github` directory together with the `workflows` folder. GitHub automatically searches this location for workflow files.

Real-World Application

DevOps engineers store workflow definitions inside this directory so GitHub Actions can automatically execute pipelines whenever repository events occur.



Task 2 – Create the CI Workflow

Objective

To create an automated GitHub Actions workflow.

Command

nano .github/workflows/ci.y

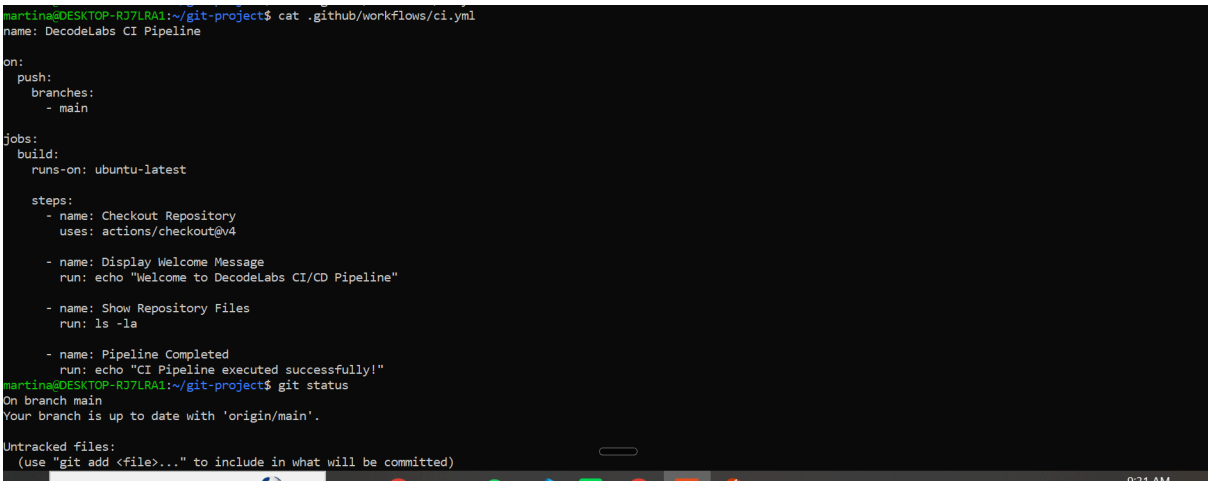
The `ci.yml` file defines the automation process using YAML syntax. It specifies when the workflow should run and what actions should be performed.

Workflow Created

name: DecodeLabs CI Pipeline

```
on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Display Welcome Message
        run: echo "Welcome to DecodeLabs CI/CD Pipeline"
```

```
    - name: Show Repository Files
      run: ls -la

    - name: Pipeline Completed
      run: echo "CI Pipeline executed successfully!"
```



Task 3 – Stage and Commit the Workflow

Objective

To save the workflow into the Git repository.

Commands Used

git status

git add .github/workflows/ci.yml

git commit -m "Add GitHub Actions CI workflow"

The workflow file was staged and committed, creating a permanent snapshot within the Git repository.

Real-World Application

Every pipeline modification is committed to Git so changes remain traceable and team members can review updates.

Task 4 – Push the Workflow to GitHub

Objective

To upload the workflow to GitHub and trigger automatic execution.

Command

git push

After pushing the repository, GitHub detected the workflow file automatically and executed the CI pipeline.

Real-World Application

This is how DevOps teams automate software builds, testing, and deployments whenever code changes are submitted.

Task 5 – Verify the Workflow

Objective

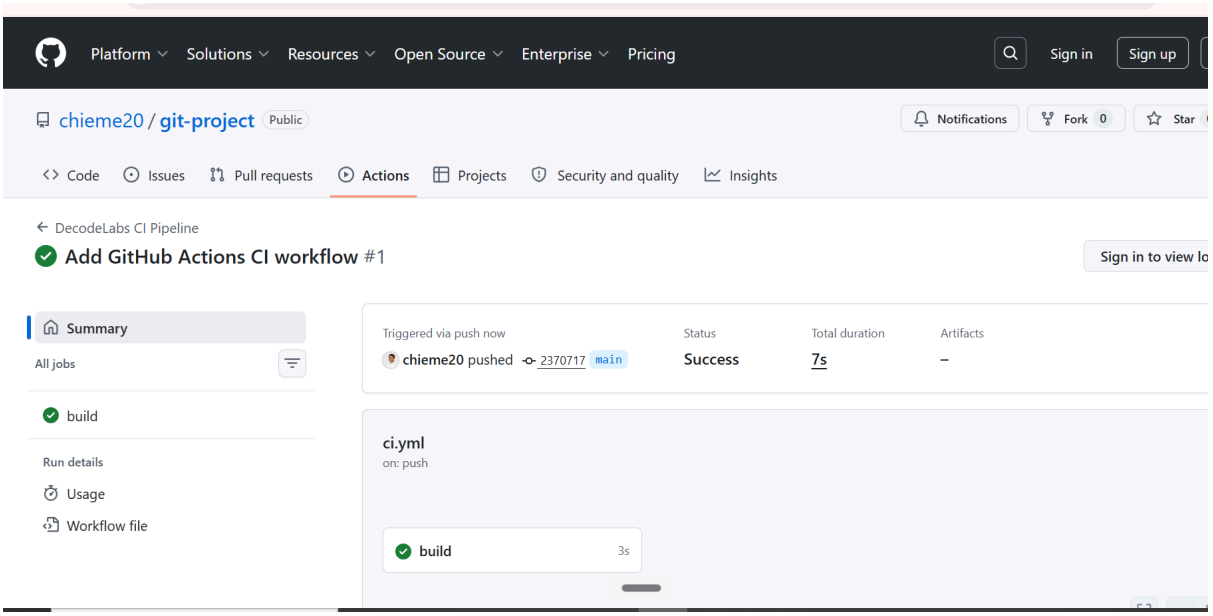
To confirm that the pipeline executed successfully.

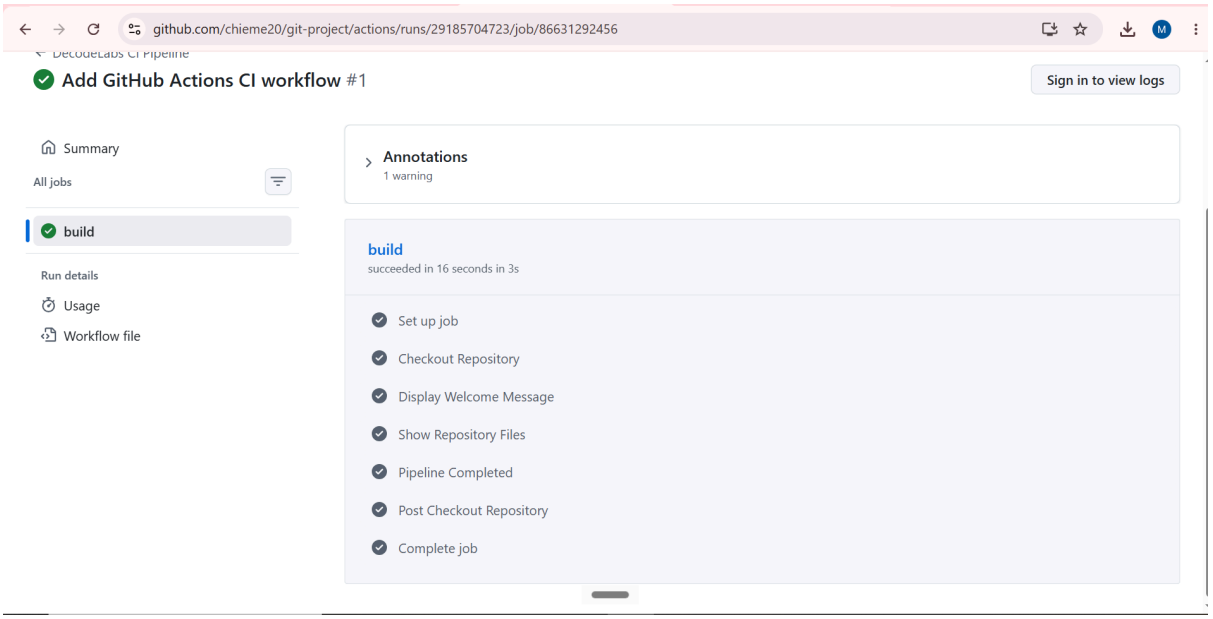
Procedure

1. Open the GitHub repository.
2. Navigate to the **Actions** tab.
3. Select the workflow run.
4. Verify that every workflow step completed successfully.

Successful Pipeline Stages

- Checkout Repository
- Display Welcome Message
- Show Repository Files
- Pipeline Completed





7. Workflow Explanation

The workflow performs the following sequence:

1. A developer pushes code to the `main` branch.
2. GitHub detects the push event.
3. GitHub launches a virtual Ubuntu runner.
4. The repository is checked out.
5. The welcome message is displayed.
6. Repository files are listed.
7. The workflow finishes successfully.

---

8. Challenges Faced

- Understanding the required directory structure for GitHub Actions.
- Learning YAML workflow syntax.
- Configuring the workflow to trigger automatically after a push.
- Verifying successful execution through the GitHub Actions interface.

---

9. Lessons Learned

Through this project, I learned how CI pipelines automate repetitive tasks during software development. I also gained practical experience creating GitHub Actions workflows, triggering automated builds, monitoring workflow execution, and understanding how automation improves consistency, efficiency, and collaboration within DevOps teams.

---

10. Conclusion

This project introduced the fundamentals of CI/CD using GitHub Actions. A functional CI pipeline was successfully created and configured to run automatically whenever changes were pushed to the GitHub repository. The successful execution of the workflow demonstrated how automation supports modern software development and DevOps practices by reducing manual effort and improving reliability.

11. References

- Git Documentation — https://git-scm.com/docs
- GitHub Actions Documentation — https://docs.github.com/actions
- DecodeLabs DevOps Industrial Training Materials